Grad Café Decoupled Microservice Platform • Module 6 Report

EN.605.601 Software Concepts • Deploy Anywhere Assignment

Student: Noella Formin • GitHub: Noella96 • Johns Hopkins University

1. Executive Summary

In Module 6, the monolithic Grad Café admissions analysis system was refactored into an enterprise-grade, decoupled 4-tier microservice architecture using Docker Compose. Long-running data ingestion, web scraping, and analytical computations were isolated from user-facing HTTP request cycles into an asynchronous worker pool backed by a durable RabbitMQ message broker. The architecture enforces non-root container security (USER 1000), queue backpressure via basic_qos(prefetch_count=1), transactional database isolation with automatic rollback, and idempotent data insertion tracked by high-water marks. The software meets rigorous assurance criteria with a **10.00/10 Pylint score** and **100.00% Pytest unit test coverage** across all services.

2. Microservice Architecture & Topology

Service	Container / Image	Port(s)	Core Responsibilities	Security & Resilience
web	`noella96/module_6-web:v1`	8080	Flask WSGI dashboard, asynchronous task offloader, REST endpoints	Non-root (USER 1000), HTTP 202 async dispatch
worker	`noella96/module_6-worker:v1`	Internal	RabbitMQ consumer, polite incremental scraper, ETL, SQL analysis	Non-root (USER 1000), prefetch=1, per-msg DB transactions
rabbitmq	`rabbitmq:3-management-alpine`	5672 15672	Durable AMQP broker, task buffering, Management UI console	Durable exchanges/queues, ping healthcheck
db	`postgres:16-alpine`	5432	Relational storage (applicants, ingestion_watermarks)	Named volume (postgres_data), pg_isready healthcheck

3. Asynchronous Queue Semantics & Reliability

Publisher Mechanics (Web Tier): When a client invokes POST /api/tasks/scrape or POST /api/tasks/recompute, the Flask application establishes a channel to RabbitMQ, idempotently declares the durable tasks direct exchange and tasks_q durable queue, and publishes a compact JSON message with delivery_mode=2 (persistent delivery). The endpoint returns immediately with HTTP 202 Accepted and a unique task tracking identifier, avoiding gateway timeouts under heavy load.

Consumer Mechanics & Backpressure (Worker Tier): The long-running worker establishes a connection to RabbitMQ and configures channel.basic_qos(prefetch_count=1). This ensures that RabbitMQ dispatches at most one unacknowledged message to the worker at a time, enforcing deterministic backpressure. The worker wraps each task execution in an isolated PostgreSQL transaction. Upon successful handler completion, the database transaction is committed and the message is positively acknowledged via basic_ack. If an unhandled exception occurs, the worker rolls back the database transaction and issues basic_nack(requeue=False), preventing poisonous infinite retry loops.

4. Database Schema, Migration & Watermark Tracking

Database initialization is automated on container launch via src/db/init.sql, which creates the applicants table, analytical indexes, and the ingestion_watermarks table. The worker tracks ingestion progress using an incremental high-water mark stored in ingestion_watermarks. All data inserts execute safe PostgreSQL SQL composition (psycopg.sql) and enforce ON CONFLICT (p_id) DO UPDATE ... to guarantee

idempotency across concurrent or repeated task executions.

5. Software Assurance Verification Evidence

A. Pytest Unit Testing Suite (100.00% Statement & Branch Coverage)

A comprehensive Pytest unit test suite was implemented across 7 test modules in module_6/tests/. All 54 unit tests pass cleanly, verifying publishers, consumers, web endpoints, database operations, ETL pipelines, and error handling edge cases. The test suite achieves **100.00% statement and branch coverage** across all 1,207 executable statements.

Source Module	Statements	Missed	Coverage
`src/db/load_data.py`	136	0	**100%**
`src/web/app/__init__.py`	14	0	**100%**
`src/web/app/routes.py`	79	0	**100%**
`src/web/models.py`	52	0	**100%**
`src/web/orm_queries.py`	91	0	**100%**
`src/web/publisher.py`	26	0	**100%**
`src/web/query_data.py`	197	0	**100%**
`src/web/run.py`	8	0	**100%**
`src/worker/consumer.py`	145	0	**100%**
`src/worker/etl/clean.py`	132	0	**100%**
`src/worker/etl/incremental_scraper.py`	75	0	**100%**
`src/worker/etl/query_data.py`	197	0	**100%**
`src/worker/etl/standardize.py`	55	0	**100%**
TOTAL (All Services)	**1,207**	**0**	**100.00%**

B. Pylint Static Code Analysis (10.00 / 10.00)

All Python source files across module_6/src/ and module_6/tests/ were evaluated using Pylint against the project .pylintrc configuration. Every single module achieved a flawless score: **Your code has been rated at 10.00/10**.

C. Continuous Integration Pipeline (GitHub Actions)

The automated CI workflow in .github/workflows/ci.yml was updated with the module_6_assurance job. The job runs in an Ubuntu runner with live PostgreSQL and RabbitMQ service containers, executing Pylint static analysis (--fail-under=10), Pytest with 100% coverage gating, Sphinx technical documentation builds, and Docker Compose configuration validation.

6. Operational Deployment & Execution Guide

The multi-service stack builds and executes cleanly on any system with Docker Desktop or Docker Engine installed. Because images are tagged and built locally, no external Docker Hub account or login is required:

1. **Start all services:** docker compose up --build
2. **Access Web Dashboard:** http://localhost:8080 (interactive analytics, search, async task buttons)
3. **Access RabbitMQ Management Console:** http://localhost:15672 (credentials: guest / guest)
4. **Stop services cleanly:** docker compose down (data preserved in named volume postgres_data)
5. **Reset and purge volumes:** docker compose down -v

Assignment Deliverable Signoff: All requirements of the JHU EN.605.601 Module 6 specification have been fully satisfied, verified, and archived in the repository. Deliverables include the containerized application, Sphinx documentation, Pytest test suite, CI workflow, and the Canvas submission package.